Santo AI OS — Developer-Facing Master Brief / Control Room

0. Current Control-Room Status

Current phase: **P0 — Foundation Slice**.

Current corrected source documents:

1. **Claude Project Instructions** — long-term architecture and philosophy.
2. **PRD v2** — product source of truth for active P0 scope.
3. **Technical Build Spec v2** — technical source of truth for P0 implementation.

Current active P0 scope:

- Primary: **Corte Santo — Daily Sales Reconciliation**
- Secondary thin validation: **Validación Facturas / XML SAT**
- Secondary thin validation: **Recibos de Utilidades**

Not active P0:

- **Employee Document Intake is not part of active P0 unless explicitly reintroduced later.**
- If referenced in older Claude Project Instructions, treat it as future/P3+ context, not active P0 scope.

Current implementation status:

- PR 1–PR 5 may start immediately.
- Business-specific workflow logic must not be finalized until Santo confirms operational inputs.
- **PR 6** may start only as Corte intake / `workflow_run` / document / task structure, without final reconciliation logic.
- **PR 7** requires confirmed Corte thresholds, severities, and reviewers.
- **PR 8** may start only as thin XML SAT validation structure, pending RFC/folder inputs.

- **PR 9** must wait for utility template, folder rules, and Sheets writeback decision.
- **PR 10** happens after prior PRs and covers hardening, tests, deploy checklist, and handoff.

Conflict rule:

If Claude Project Instructions conflict with PRD v2 or Technical Build Spec v2 on active P0 scope, **PRD v2 and Technical Build Spec v2 override for P0**. Claude Project Instructions remain the long-term architectural constitution, not the final authority on corrected P0 workflow selection.

1. Project North Star

Santo AI OS is Santo Group's internal operating brain.

It is not a generic chatbot, not a collection of random scripts, and not only a dashboard. It is the layer that connects Santo's people, documents, workflows, approvals, exceptions, reports, and decisions.

The system should help Santo remember:

- how work happens
- who owns each process
- what documents are needed
- where those documents live
- what is missing
- what is blocked
- what needs approval
- what action should happen next
- what happened before
- what failed
- who approved what

The long-term goal is to create a company-wide operational intelligence layer for Santo Group.

The system should eventually support:

- Admin
- HR
- Payroll
- Accounting
- Fiscal
- Legal
- Operations
- Restaurant Operations
- Purchasing
- Inventory
- Marketing
- Financial Reporting
- Management Reporting
- Partner / Investor Reporting

The goal is compounding operational memory. Every workflow added to the system should make Santo smarter, more consistent, easier to manage, and less dependent on knowledge living only in people's heads, WhatsApp messages, scattered Drive folders, emails, or spreadsheets.

2. Current Source-of-Truth Hierarchy

The project currently has three source-of-truth layers.

A. Claude Project Instructions

These define the long-term Santo AI OS vision, architecture, safety principles, stack, skill rules, progressive context loading, P0-to-P10 roadmap, and high-level execution philosophy.

They are the long-term architectural constitution.

However, they contain some older optional P0 workflow references. For active P0 scope, PRD v2 and Technical Build Spec v2 override Claude Project Instructions when there is a conflict.

B. PRD v2

This is the product source of truth for active P0.

It defines:

- what P0 is
- what P0 is not
- the current three active P0 workflows
- Agent Mail's role
- product requirements
- success criteria
- open business inputs needed from the Santo team
- roadmap after P0

Current active P0 workflows are:

1. Corte Santo — Daily Sales Reconciliation
2. Validación Facturas / XML SAT
3. Recibos de Utilidades

Employee Document Intake is not active P0 unless explicitly reintroduced later.

C. Technical Build Spec v2

This is the technical source of truth for the P0 build.

It defines:

- PR-by-PR implementation sequence
- repo structure
- `santo_context.md`
- `skill_template.md`
- `script.py` contract

- schema requirements
- `workflow_runs`
- `email_messages`
- module config schema
- idempotency / `source_hash`
- default `requires_review` behavior
- Agent Mail routing
- RLS/access pattern
- Drive folder map rules
- scheduler guidance
- acceptance criteria before merge

D. Future Operational Addendum

This does not exist yet.

It will be created only after the Santo admin team answers operational questions.

This addendum is operational/configuration only. It is not a new technical source of truth and should not replace PRD v2 or Technical Build Spec v2.

It should contain confirmed business inputs such as:

- Drive folder URLs and hierarchy
- file naming conventions
- Corte Santo thresholds
- exception severity rules
- reviewer/owner mappings
- restaurant/entity/RFC mappings
- Agent Mail routing convention
- Utility receipt template columns and marking rules

- XML SAT export examples / fixture rules

Until this operational addendum exists, these business rules must not be hardcoded.

3. Current Build Philosophy

The full Santo AI OS vision must remain visible, but the build must be phased.

P0 does not attempt to build the entire Santo AI OS.

P0 proves the operating model through a narrow foundation slice:

- one implementation domain
- one primary workflow
- two thin secondary validation workflows
- one dashboard path
- one notification/intake channel
- one approval/review model
- one registry spine
- one shared command-handler architecture

The key principle:

P0 proves the operating model. Later phases expand the operating system.

Do not confuse the North Star with the current build phase.

If a feature belongs to the long-term Santo AI OS vision but is not required for the current phase, it should be documented in the roadmap instead of being built immediately.

4. Current Implementation Domain

The first implementation domain is:

Admin / HR / Payroll / Accounting / Fiscal.

This is the starting point because these workflows are currently documented and operationally important.

Initial workflow owners:

- Mich — HR and payroll operations
- Abraham — Accounts payable / CxP
- Fernanda — Fiscal and tax compliance
- Manuela — Accounting manager / accounting control

Future departments should not be automated until their workflows are documented and the P0 foundation works.

However, all architecture, naming, schema, module structure, and trigger design must allow future departments to plug in without rebuilding the foundation.

5. Current P0 Workflow Scope

The current P0 workflow scope is defined by PRD v2 and Technical Build Spec v2.

Primary P0 Workflow

1. **Corte Santo — Daily Sales Reconciliation**

This is the primary P0 workflow because it proves:

- daily recurring operations
- restaurant/unit-level data
- reconciliation logic
- exception detection
- dashboard visibility
- summary generation
- management reporting
- workflow run tracking
- watchdog/failure alerts
- human review flow

P0 Corte Santo must remain narrow:

- one restaurant/unit or business unit first
- one controlled source path
- one reconciliation pattern
- one exception model
- one dashboard status path
- Agent Mail intake/notification
- configurable rules, not hardcoded assumptions

It must not become the full financial reporting engine in P0.

Thin Secondary Validation Workflows

2. Validación Facturas / XML SAT

Purpose in P0:

- prove the foundation can support fiscal workflows
- process exported XMLs or validation artifacts
- group and track by RFC/month/type
- create exceptions and review needs

P0 must not automate SAT login, FIEL, IDSE, IMSS, government portals, or credential-sensitive fiscal actions.

3. Recibos de Utilidades

Purpose in P0:

- prove document matching and Drive filing/tracking
- match receipts to the utility template
- register status and exceptions
- defer Google Sheets writeback unless explicitly confirmed

These two secondary workflows are thin validation workflows. They must reuse the same foundation and must not compete with Corte Santo as the main P0 build.

Important scope correction:

Employee Document Intake is not part of the active P0 build. It may appear in older Claude Project Instructions as an optional secondary workflow, but the corrected PRD v2 and current P0 final scope replace that with XML SAT and Recibos de Utilidades. Treat Employee Document Intake as a future/P3+ workflow unless Alonso explicitly reintroduces it into P0.

6. Current P0 Product Definition

The current P0 product definition comes from PRD v2.

P0 is the first buildable version of Santo AI OS.

It must prove the operating model, not the full company-wide vision.

P0 includes:

- Supabase/Postgres foundation
- workflow registry
- operating domain registry
- workflow runs
- documents
- tasks
- exceptions
- approvals/reviews
- events
- watchdog logs
- email_messages
- drive_folder_map
- dashboard shell
- Agent Mail intake and notification
- shared command handler
- Corte Santo primary workflow

- XML SAT thin validation workflow
- Recibos de Utilidades thin validation workflow

P0 must not include:

- production WhatsApp
- full financial reporting engine
- full restaurant operations automation
- SAT/FIEL/IDSE/IMSS automation
- bank actions
- payroll execution
- autonomous approvals
- full multi-department expansion
- hardcoded Drive paths
- hardcoded thresholds
- hardcoded reviewer logic
- hardcoded restaurant/entity assumptions

The P0 rule is:

Build the smallest reliable system that proves the Santo AI OS pattern.

7. Agent Mail Role in P0

Agent Mail is part of P0.

The intended email identity is:

`os@santo.com`

or equivalent.

Agent Mail is the controlled intake and notification channel for the OS.

It is used so the system does not need broad access to Manuela's personal/work inbox.

P0 Agent Mail responsibilities:

- receive workflow inputs and attachments
- create `email_messages`
- link classified emails to `workflow_runs`
- store attachment metadata as `documents`
- trigger or prepare workflow processing
- send summaries, alerts, and review requests
- create events for inbound and outbound email activity

Agent Mail must not:

- become the source of truth
- approve high-risk actions
- read unrelated personal/work inbox content
- silently ignore ambiguous messages
- expose sensitive payroll, bank, fiscal, or legal data in email bodies

Unknown or ambiguous emails must use:

`requires_review`

not:

`ignored`

`ignored` is only allowed when the email is definitively non-workflow by a confirmed rule, such as a known newsletter, spam sender, or explicitly excluded source.

8. Trigger Layer

All user-facing triggers must eventually pass through one shared command handler.

Trigger sources may include:

- dashboard buttons

- Agent Mail / email
- scheduled jobs
- WhatsApp later
- future forms or system events

P0 production trigger channels:

- dashboard
- Agent Mail
- scheduled jobs if needed

P0 non-production / future trigger channels:

- WhatsApp

WhatsApp must be designed for later, but not forced into P0.

Do not build separate automation systems for each channel.

The command flow should be:

user interface → command handler → permission check → workflow module → database update → notification/output → event/audit log.

9. Technical Source of Truth for Implementation

The implementation should follow the Technical Build Spec v2.

The Technical Build Spec v2 defines:

- repo structure
- PR sequence
- Supabase schema expectations
- `santo_context.md`
- `skill_template.md`
- `script.py` execution contract

- `workflow_runs`
- `email_messages`
- module `config.example.json`
- idempotency strategy
- `source_hash`
- default `requires_review` behavior
- Agent Mail routing
- RLS pattern
- Drive folder map rules
- scheduler decision
- merge acceptance criteria

Foundation PRs may start immediately:

- PR 1 — repo, templates, `santo_context.md`, `skill_template.md`, env examples
- PR 2 — Supabase schema, RLS, seed placeholders, `workflow_runs`, `email_messages`
- PR 3 — dashboard shell, auth, roles
- PR 4 — shared command handler, events, watchdog
- PR 5 — Agent Mail polling/intake and metadata logging

Business-specific workflow logic must not be finalized until the Santo team provides confirmed inputs.

Business-specific logic includes:

- Drive folder URLs and naming rules
- Corte Santo thresholds
- exception severity rules
- reviewer/owner mappings

- required attachments
- restaurant/entity/RFC mappings
- Agent Mail routing convention
- Utility receipt template columns and marking rules
- XML SAT export examples / fixtures

These inputs will become the future operational addendum.

10. Skill System Rules

Every workflow is a self-contained skill/module.

A workflow module should include:

- `skill.md`
- `script.py`
- `config.example.json`
- tests
- fixtures
- references to shared clients/utilities

Every skill must follow the standard skill execution contract.

Each skill must define:

- objective
- workflow owner
- inputs
- outputs
- required environment variables
- data model dependencies
- risk level

- execution environment
- approval/review points
- trigger channels
- input schema
- output schema
- dry-run mode
- idempotency key
- logging behavior
- exception behavior
- watchdog behavior
- test fixture
- manual fallback
- rollback procedure

Every `script.py` must be able to answer:

- What did it receive?
- What did it change?
- What did it create?
- What did it skip?
- What failed?
- What needs human review?
- What was logged?
- Can it safely run twice?

11. DRY / MECE Skill Registry Rule

Before creating any new skill, the developer must check:

- the Workflow Registry
- the existing skill folder index
- existing module purposes
- existing inputs and outputs

The developer must confirm:

1. No existing skill already covers this function.
2. No existing skill can be parameterized to cover this case.
3. The new skill does not overlap with an existing skill's stated purpose, inputs, or outputs.

If an existing skill can be extended or parameterized, the default action is to extend the existing skill, not create a new one.

Every new skill must be registered in the Workflow Registry before the first commit.

A skill that is not in the registry does not exist for Santo AI OS purposes.

This rule prevents Santo AI OS from becoming a collection of duplicate scripts.

12. Progressive Context Loading

The project uses progressive context loading.

`santo_context.md` should remain lightweight.

It contains only:

- company overview
- brand list
- operating domains
- current implementation phase
- active pilot workflow
- key naming conventions
- high-level rules

It must not become a full company encyclopedia.

Detailed domain context should live in separate context files, such as:

- `context_admin.md`
- `context_accounting.md`
- `context_fiscal.md`
- `context_hr.md`
- `context_legal.md`
- `context_restaurant_ops.md`

Every skill loads:

1. `santo_context.md`
2. only the relevant domain context file
3. its own `skill.md`
4. its own `config.example.json` / confirmed config

Examples:

- Corte Santo should load `santo_context.md` + accounting/admin context.
- XML SAT should load `santo_context.md` + fiscal context.
- HR/IDSE workflows should load `santo_context.md` + HR context.

This prevents token/context bloat as the OS scales.

Adding a new department should mean adding a new domain context file, not expanding the master context endlessly.

13. Current Open Operational Inputs

The following inputs are still pending from the Santo admin team.

These are not the developer's responsibility to invent.

They must be answered by Manuela, Abraham, Fernanda, Mich, or the relevant operational owner.

Pending inputs:

1. Confirmed restaurant names and short codes.
2. Legal entity / RFC mapping per restaurant.
3. Workflow owners and reviewers per exception type.
4. Exact Drive folder URLs and hierarchy.
5. Drive read/write permissions per folder.
6. Naming conventions for Corte, XML, Utilidades, reports, and evidence.
7. Required Corte Santo attachments per restaurant/source.
8. Corte Santo reconciliation thresholds.
9. Exception severity rules.
10. Agent Mail routing convention.
11. Utility receipt template structure.
12. Utility receipt row marking/color rules.
13. Whether Utility Receipts may write back to Google Sheets in P0.
14. At least one sanitized/anonymized real XML export from MiAdminXML for fixtures.
15. At least one sanitized/anonymized Corte input example for fixtures if possible.

These answers will become the future operational addendum.

Until confirmed, all uncertain business rules must default to:

`requires_review`

and never to:

`completed`.

13A. Pending Stream — Operational Addendum

The Santo admin team responses are pending. When received, they must be processed and converted into the P0 Operational Addendum.

This addendum should define:

- Drive folder URLs and hierarchy
- file naming conventions
- Corte Santo thresholds
- required attachments
- exception severity rules
- reviewer/owner mappings
- restaurant/entity/RFC mappings
- Agent Mail routing convention
- Utility receipt template columns and marking rules
- XML SAT export examples / fixture rules

These answers unblock **PR 6–PR 9**.

These rules must not be invented or hardcoded.

14. Technical Architecture Summary

The current technical architecture is:

Dashboard / Agent Mail / Scheduled Jobs / future WhatsApp
→ Shared Command Handler
→ Permission Check
→ Workflow Module
→ Supabase/Postgres
→ Notification / Output
→ Events + Watchdog

Core stack:

- Supabase/Postgres = source of truth
- Next.js or equivalent = dashboard

- Agent Mail / Gmail = controlled intake and notification channel
- Python scripts = workflow execution layer
- Claude = reasoning, classification, drafting, summarization
- Composio = connector layer only
- Google Drive = document/evidence storage
- GitHub Actions or Railway = scheduler/worker
- WhatsApp Business API = later operational command channel

Important rule:

Third-party tools are pipes, not the brain.

Composio, Gmail, Drive, WhatsApp, Twilio, 360dialog, Slack, or any future connector must not become:

- the source of truth
- the approval model
- the audit trail
- the security model

Supabase/Postgres owns state, approvals, workflow memory, and audit logs.

15. Core Data Model

The P0 data model should include at minimum:

- `operating_domains`
- `workflows`
- `workflow_runs`
- `documents`
- `people`
- `vendors`

- `restaurants`
- `legal_entities`
- `tasks`
- `exceptions`
- `approvals` or review records
- `watchdog_log`
- `events`
- `drive_folder_map`
- `email_messages`
- access tables such as `profiles`, `user_domain_access`, `workflow_access`

Important P0 schema additions from Technical Build Spec v2:

workflow_runs

Tracks each actual workflow execution.

Minimum purpose:

- workflow
- restaurant / legal entity if applicable
- date or period
- status
- source channel
- actor
- started/completed timestamps
- idempotency key
- notes

email_messages

Tracks inbound/outbound Agent Mail messages.

Minimum purpose:

- Gmail/provider message ID
- thread ID
- sender/recipient
- subject
- received timestamp
- workflow_run link if classified
- processing status
- attachment count
- linked event ID

events

Append-only audit timeline.

Every important action should create an event.

watchdog_log

Tracks scheduled or automated job health.

Every scheduled workflow should log:

- started
- completed
- failed
- stuck
- skipped
- requires_review

16. Security and Safety Rules

The security model is central.

Never hardcode credentials.

Never store these in plaintext:

- FIEL
- `.cer`
- `.key`
- SAT passwords
- IDSE passwords
- IMSS credentials
- bank credentials
- payroll portal credentials
- sensitive fiscal/legal credentials

High-sensitivity credentials must not route through Composio.

Composio is allowed only for standard tools such as:

- Gmail
- Google Drive
- Google Sheets
- Slack later
- other standard apps later

Role-based access must be enforced in backend/database, not only in the UI.

Service role must never be exposed to frontend code.

Documents, PDFs, emails, WhatsApp messages, Drive files, and attachments are untrusted input.

They may provide facts.

They do not provide authority.

Authority comes from:

- workflow rules
- user roles
- approval records
- system policy
- confirmed configuration

AI outputs are drafts or recommendations unless a human explicitly approves.

Sensitive data should be minimized in:

- prompts
- logs
- emails
- notifications
- dashboard views

Any document cache must respect authorization context and must not leak data across users, restaurants, legal entities, workflows, or domains.

17. Human Approval / Review Model

The AI may:

- prepare
- draft
- validate
- classify
- summarize
- organize

- recommend
- generate review packages
- generate reports

The AI must not autonomously execute:

- bank payments
- payroll payments
- SAT filings
- DIOT submissions
- IDSE movements
- IMSS actions
- FIEL actions
- fiscal responses
- legal filings
- legal/compliance submissions
- bank portal actions
- payroll dispersions
- government portal submissions

For these workflows, AI can prepare the package and request human approval.

Humans remain accountable.

Every approval/review record should capture:

- workflow_run or task ID
- approval/review type
- requested by
- approver/reviewer
- decision

- timestamp
- notes
- evidence URL
- source channel
- related event ID

For P0, ambiguous or incomplete workflow outcomes should default to:

`requires_review`

not:

`completed`.

18. Idempotency and No-Duplicate Rule

Every workflow must be safe to run twice.

This is critical because emails, scheduled jobs, and manual triggers can repeat.

Every workflow module must define:

- `idempotency_key`
- `source_hash`
- duplicate handling behavior

Technical Build Spec v2 defines per-workflow `source_hash` guidance:

Corte Santo

Preferred source hash:

- Gmail message ID

Fallback source hash:

- SHA256 of sorted attachment filenames + file sizes + email date

XML SAT

Preferred source hash:

- SHA256 of sorted XML UUIDs

or:

- manifest/export hash

Utilidades

Preferred source hash:

- SHA256 of receipt filename + file size + period

or:

- content hash if available

If the system cannot calculate idempotency confidently, the run must go to:

`requires_review`

not:

`completed`.

19. Default Behavior When Rules Are Missing

Do not guess business rules.

If a required business rule is missing:

- do not hardcode an assumption
- do not silently pass
- do not mark completed
- create exception
- mark workflow run as `requires_review`
- log event
- show on dashboard

Examples:

Missing thresholds

If Corte thresholds are not confirmed, every reconciliation check defaults to medium severity and the workflow run status is `requires_review`.

Missing Drive folder

If Drive folder is not confirmed, do not write to unknown folder. Create placeholder, create exception, mark as `requires_review`.

Missing reviewer

If reviewer is not confirmed, use default owner only if configured. Otherwise mark as `requires_review`.

Unknown email

If email cannot be confidently routed to Corte, XML, or Utilidades, set email status to `requires_review`.

Do not use `ignored` unless a confirmed rule says the email is definitely non-workflow.

20. Drive Folder Strategy

For P0, do not reorganize Santo's Drive.

Use the current folder structure.

But "mirror current Drive" means:

- map existing folders into `drive_folder_map`
- store folder IDs/URLs in the database/config
- do not hardcode Drive paths in code
- do not move or restructure everything in P0
- do not write into unconfirmed folders

Future phases may create a cleaner AI OS-controlled Drive structure, but P0 should avoid operational disruption.

The admin team must confirm:

- folder URLs
- hierarchy
- read/write permissions
- folder purpose
- naming rules
- folders that should never be touched

21. Developer Implementation Context

The implementation developer should build from:

- PRD v2
- Technical Build Spec v2
- Claude Project Instructions
- Future Operational Addendum once available

The developer's work should be reviewed against:

- scope discipline
- schema alignment
- RLS/security
- no hardcoded business assumptions
- idempotency
- auditability
- test coverage
- fallback behavior
- acceptance criteria

The developer should not invent business rules.

Business-specific logic must wait for confirmed operational inputs, including:

- Drive folder URLs
- naming conventions
- Corte Santo thresholds
- required attachments
- exception severity rules
- reviewer/owner mappings
- restaurant/entity/RFC mappings
- Agent Mail routing convention
- Utility receipt template rules
- XML SAT fixture examples

The project should be implemented by phase, following the PR sequence and acceptance criteria in the Technical Build Spec v2.

22. Roadmap P0–P10

P0 — Foundation Slice

Goal: prove the Santo AI OS operating model with one primary workflow and two thin secondary validation workflows.

Primary workflow:

- Corte Santo — Daily Sales Reconciliation

Thin secondary validation workflows:

- Validación Facturas / XML SAT
- Recibos de Utilidades

P0 success means:

- Agent Mail receives and logs workflow inputs.
- Dashboard shows workflow runs, Corte status, exceptions, and review needs.

- Supabase/Postgres stores workflow memory.
- Corte Santo runs for one restaurant/unit first.
- XML SAT and Utilidades prove the same foundation can support other workflows.
- Missing business rules default to `requires_review`.
- No high-risk action is autonomous.

P1 — Stabilize Corte Santo

Goal: make Corte Santo reliable, boring, and repeatable for one restaurant/unit.

Focus:

- real source examples
- clean parsing
- reconciliation rules
- dashboard usability
- Agent Mail notifications
- manual review flow
- audit quality
- test coverage
- failure handling

P2 — Scale Corte Santo Across Restaurants

Goal: expand Corte Santo to multiple restaurants without rewriting the module.

Focus:

- restaurant-specific config
- Drive folder maps per unit
- source mapping by restaurant
- thresholds by source/check type

- reviewer mapping
- bulk status dashboard
- exception summaries by restaurant

Rule:

Do not create one custom Corte script per restaurant. Parameterize one reusable module.

P3 — Build Additional Admin/Fiscal Workflows

Goal: build fuller versions of secondary/admin workflows after P0 proves the foundation.

Possible P3+ workflows:

- XML SAT Validation full workflow
- Recibos de Utilidades full workflow
- Employee Document Intake
- Supplier Follow-up
- Payment Evidence Filing
- Duplicate Payment Detection
- Digital Payroll Receipt Signature Pilot

Important:

Employee Document Intake belongs here unless explicitly reintroduced into P0 later. It should not be treated as part of active P0.

All workflows must reuse:

- Workflow Registry
- Document Registry
- Task Registry
- Exception Log
- Approval/Review Log
- Watchdog Log

- Event Log
- Agent Mail
- Dashboard
- Shared Command Handler

P4 — Productionize the Command Layer

Goal: expand interaction channels safely.

Add:

- dashboard commands
- email commands
- WhatsApp commands
- role-based command permissions
- command audit log
- safe command templates

WhatsApp starts only with safe commands, such as:

- show my pending tasks
- show Corte status
- show exceptions this week
- run workflow check
- send Corte exceptions
- show missing documents

No high-risk approvals through WhatsApp until authentication, role checks, approval records, and audit logs are strong.

P5 — Query Mode

Goal: allow users to ask natural-language questions over structured Santo data.

Examples:

- Which restaurants have Corte exceptions?
- What needs Manuela's review?
- Which invoices are missing XMLs?
- Which tasks failed recently?
- What is blocked this week?
- Which units have unresolved reconciliation issues?

Rule:

Query Mode answers from structured Supabase/Postgres data first.

The AI should not guess when the database already contains the status.

P6 — Action Mode

Goal: allow the system to prepare work, drafts, reports, and review packages.

Examples:

- prepare Corte daily summary
- draft follow-up for unresolved reconciliation issues
- prepare missing XML report
- generate supplier follow-up drafts
- generate accounting exception summary
- prepare partner update package

Rule:

AI may prepare and draft. Humans approve high-risk actions.

P7 — Accounting / Fiscal Control Buildout

Goal: expand into serious accounting and fiscal workflows.

Possible workflows:

- IDSE Acuse Follow-up

- Payment Execution Control
- Tax Determination Prep
- DIOT Prep
- Monthly P&L Prep
- Tips Dispersion Control
- Payroll Receipt Signature Control

Focus on preparation, validation, tracking, and control.

Do not autonomously file or pay.

P8 — Department Expansion Framework

Goal: add new departments without rebuilding the foundation.

Before automating a new department:

1. Collect workflows.
2. Map owners.
3. Map tools.
4. Map documents.
5. Add workflows to registry.
6. Classify local / remote / hybrid.
7. Score risk.
8. Define approvals.
9. Estimate time savings.
10. Choose one safe pilot.
11. Build one module.
12. Scale only after it works.

Future departments:

- Legal

- Operations
- Restaurant Operations
- Purchasing
- Inventory
- Marketing
- Financial Reporting
- Management Reporting
- Partner / Investor Reporting

P9 — Reporting and Presentation Engine

Goal: turn operational memory into leadership-ready reporting.

Outputs:

- weekly operations summary
- monthly accounting summary
- missing documents report
- P&L exception report
- supplier issue report
- workflow bottleneck report
- partner update deck
- restaurant-level exception summary
- Corte Santo trend summary
- reconciliation exception report

P10 — Full Santo AI OS

Goal: complete company-wide operating layer.

Capabilities:

- company memory
- workflow automation
- document intelligence
- approval/review control
- exception management
- dashboard interface
- email interface
- WhatsApp interface
- query mode
- action mode
- reporting engine
- department expansion system
- audit trail
- failure monitoring
- partner / investor reporting

Final architecture:

- Supabase/Postgres = source of truth
- Dashboard = primary staff interface
- Agent Mail = communication and trigger channel
- WhatsApp = operational command channel
- Composio = connector layer only
- Claude = reasoning/action layer
- Python skills = modular execution layer
- GitHub Actions / Railway / VM / local machine = execution environments
- Watchdog = reliability layer

- Approvals/reviews = human control layer
- Events = audit/history layer

23. Recommended Chat / Review Structure Going Forward

This project should be split into dedicated review/context streams to avoid context overload.

Stream 1 — Santo AI OS / Master Brief / Control Room

Role:

- strategic memory
- source-of-truth summary
- orientation for new work
- roadmap alignment

Stream 2 — P0 Build Audit

Role:

- strict technical review
- PR audit
- merge-readiness review
- scope/security/RLS/idempotency check

Use this whenever there is:

- PR summary
- code diff
- implementation report
- Claude Code review
- branch status
- bug fix

Default question:

“Is this aligned with the P0 PRD v2 and Technical Build Spec v2? What passes, what fails, and what must be fixed before merge?”

Stream 3 — P0 Admin Inputs to Config

Role:

- translate Manuela/Abraham/Fernanda/Mich answers into technical config
- build operational addendum
- convert business rules into:
    - folder map
    - thresholds
    - reviewer map
    - naming conventions
    - restaurant/RFC mappings
    - utility template rules
    - Agent Mail routing rules

This stream should handle messy human answers and turn them into clean developer-ready config.

Stream 4 — Future Roadmap P1–P10

Role:

- long-term planning
- future departments
- WhatsApp
- Query Mode
- Action Mode
- investor/partner reporting

- legal ops
- inventory
- purchasing
- reporting engine

This prevents future ambition from contaminating P0 execution.

24. How to Start Any New Chat or Review Thread

Every new Santo AI OS chat/review thread should begin with:

“Use the Santo AI OS Master Brief as context. Your role in this chat is: [ROLE]. Stay inside that role. Do not reopen the whole strategy unless it affects this role.”

Then paste the relevant brief or section.

For P0 PR audits, also attach or reference:

- PRD v2
- Technical Build Spec v2
- PR/diff/summary
- any Claude Code review

For admin inputs, also attach or paste:

- team answers
- Drive links/folder rules
- threshold answers
- reviewer mappings
- templates or screenshots

25. Immediate Next Steps

Current recommended next steps:

1. Use this Master Brief as the source-of-truth summary.
2. Use PRD v2 and Technical Build Spec v2 as the implementation source of truth.
3. Start PR 1–PR 5.
4. Allow PR 6 only as Corte intake / `workflow_run` / documents / tasks structure, without final reconciliation logic.
5. Do not allow PR 7 until Corte thresholds, severities, required attachments, and reviewers are confirmed.
6. Allow PR 8 only as thin XML SAT validation structure, pending RFC/folder inputs and sanitized XML fixtures.
7. Do not allow PR 9 until utility template columns, folder rules, marking/color rules, and Sheets writeback scope are confirmed.
8. Collect admin team answers for the operational addendum.
9. Do not hardcode Drive paths, thresholds, reviewers, restaurant mappings, email routing, `source_hash` rules, or business logic.
10. Convert admin team answers into the operational addendum.
11. Review PRs against the Technical Build Spec v2 before merge.
12. Keep future roadmap ideas separate from active P0 implementation.

26. Current Golden Rule

The Santo AI OS should not become a pile of scripts.

Every workflow must connect back to:

- Workflow Registry
- Document Registry
- Task / Workflow Run Registry
- Exception Log
- Approval / Review Log
- Watchdog Log

- Event Log
- Dashboard
- Agent Mail
- Shared Command Handler

The goal is not isolated automation.

The goal is compounding company-wide operational memory.